

Algorithmes Génétiques

Aperçu

- **Introduction.**
- Représentation d'un problème.
- Constituants de base des algorithmes génétiques : sélection, opérateurs d'évolution et remplacement.
- Quelques considérations.
- La programmation génétique.

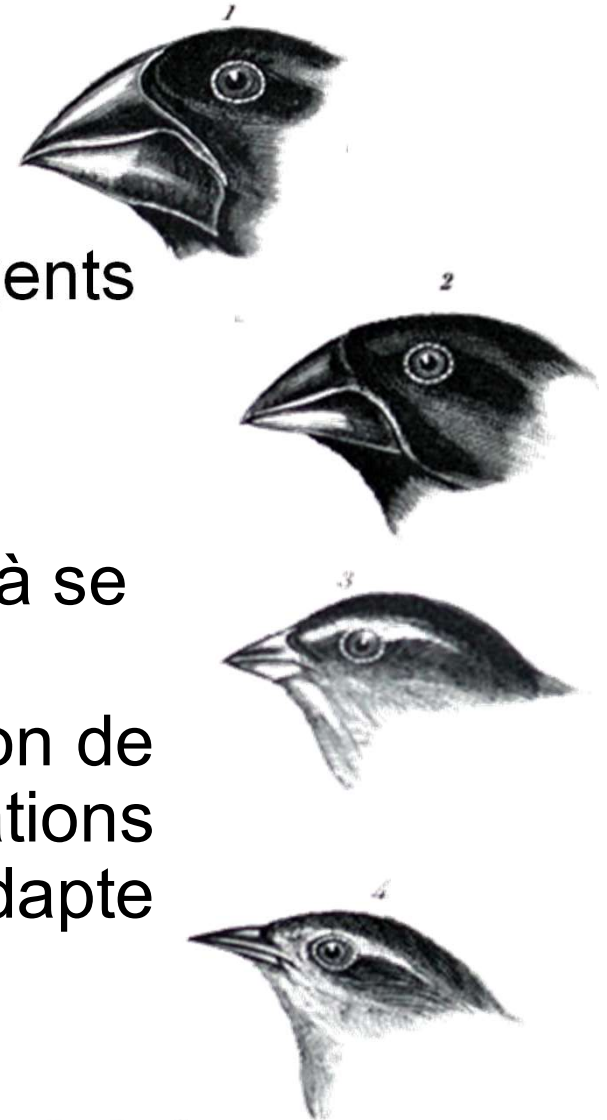
Certains transparents présentés dans ce chapitre s'inspirent de ceux du Prof. J-C. Latombe (Université de Stanford) et ont été adaptés à notre contexte, ceci avec l'autorisation de l'auteur.

Algorithmes évolutionnaires

- **Fondements biologiques**
 - C. Darwin (1809-1882) A. R. Wallace (1823-1913): sélection naturelle
 - G. Mendel (1822-1884) : hérédité des caractères « génétiques »
 - T. H. Morgan (1866–1945) : théorie chromosomique
- **Algorithmes génétiques** *Genetic Algorithms*
 - GA J. Holland (1962, 1975, Ann Arbor)
- **Programmation génétique** *Genetic Programming*
 - GP J. Koza (1989, Palo Alto)
- **Stratégies d'évolution** *Evolution Strategies*
 - ES I. Rechenberg et H.-P. Schwefel (1965, Berlin)
 - codage réel, mutation gaussienne
- **Programmation évolutionnaire** *Evolutionary Programming*
 - EP L. Fogel (1962, San Diego)

Principe général de l'évolution darwinienne

- Le contexte de l'évolution est une **population** (d'organismes, d'objets, d'agents ...) qui survivent un temps limité puis meurent. Certains produisent une **descendance** pour les générations suivantes. Les '**mieux adaptés**' tendent à se reproduire plus.
- Sur plusieurs **générations**, la composition de la population change. Lors des générations successives, « l'espèce » évolue et s'adapte en quelque sorte aux conditions.



Principe général de l'évolution

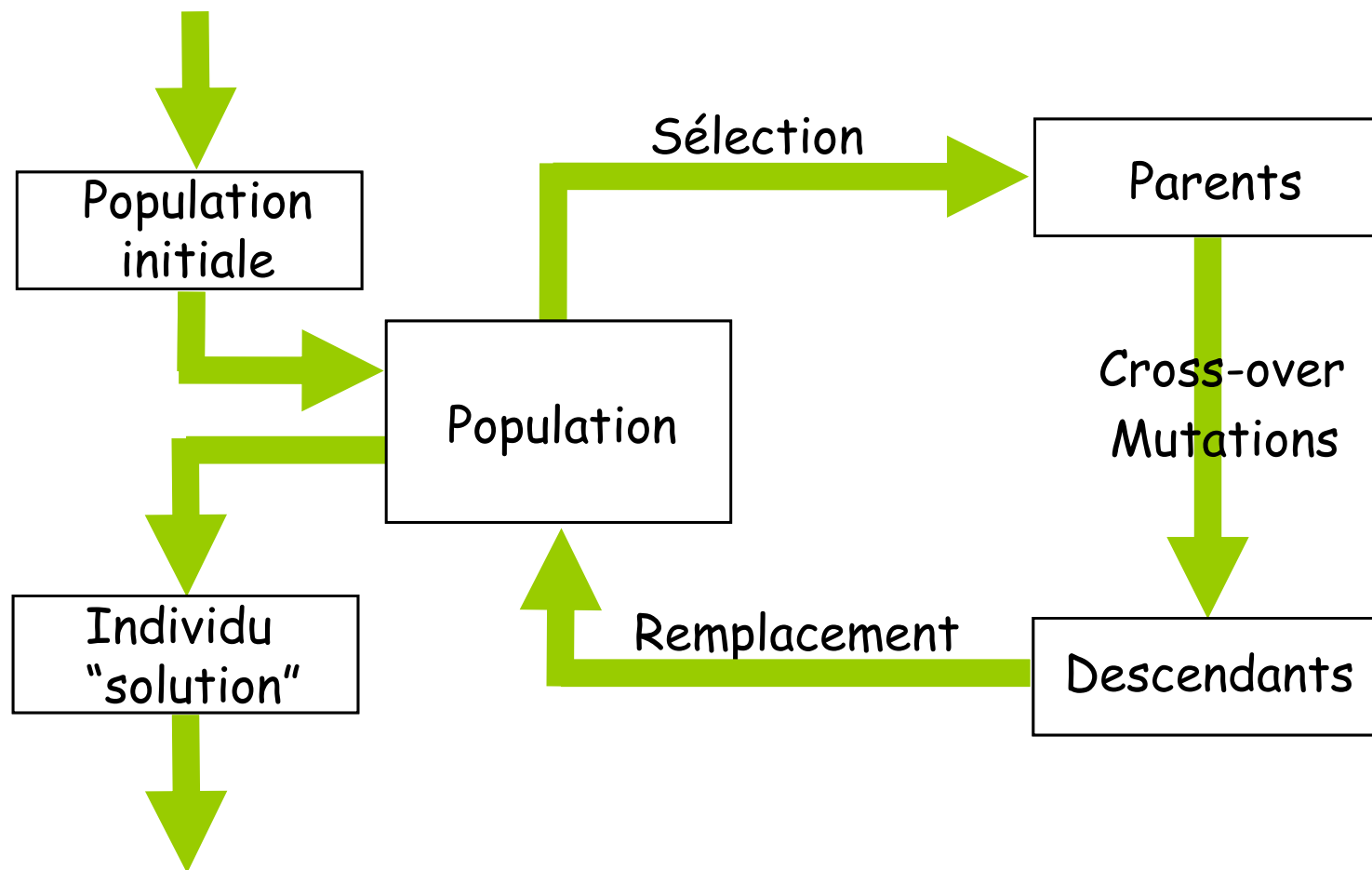
- Hérité
– Les descendants sont **similaires** aux parents
- Variabilité
– Des changements aléatoires « perturbent » l'hérité
– Des recombinaisons aléatoires augmentent la diversité
- Sélection
– Les organismes les plus adaptés à leur environnement sont susceptibles d'avoir plus de descendants

Ces trois processus sont réunis au sein d'une boucle générationnelle ...

Algorithmes Génétiques: méthode d'exploration aléatoire bio-inspirée

- Sur le plan algorithmique, les AGs sont caractérisés par
 - Codage des solutions potentielles (génotypes des individus)
 - Exploration multiple (population)
 - La fonction à optimiser permet l'évaluation des individus (performance ou *fitness*)
 - Règles de transition probabilistes (mécanismes de sélection / reproduction / mutation).
- On cherche une bonne solution à partir d'une population de solutions candidates
 - Tant qu'une bonne solution n'est pas trouvée, la population est soumise à une évolution dirigée

Principe du cycle d'évolution



Algorithme génétique

créer population initiale

faire

évaluer tous les individus (avec fct fitness)

faire

sélectionner parents

créer descendants par recombinaison

faire muter descendants

jusqu'à taille population

remplacer tous les individus par les descendants

jusqu'à critère d'arrêt

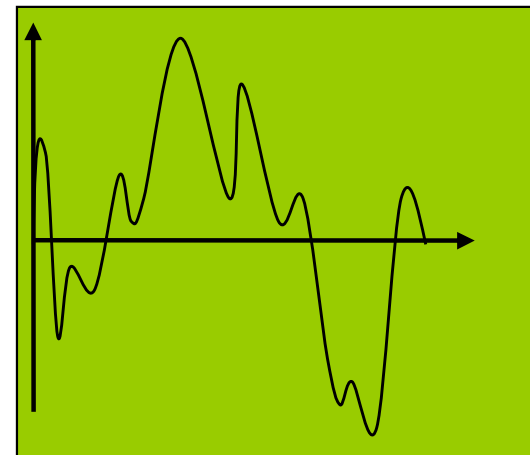
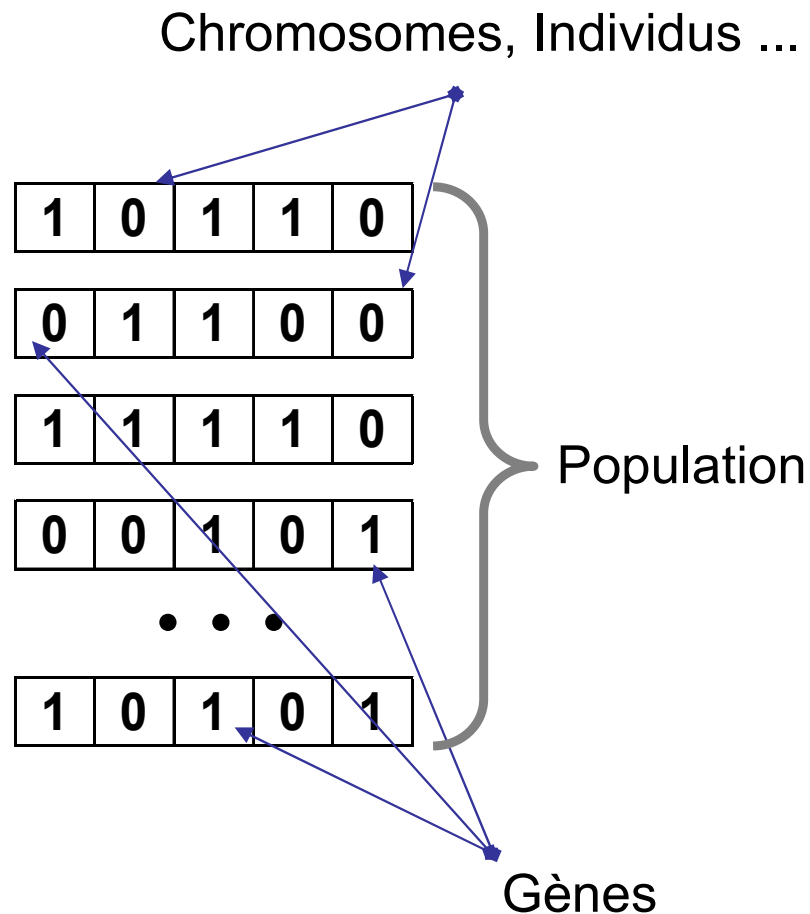
Terminologie

Génotype : configuration de la chaîne binaire

Phénotype : ce que le génotype représente
« réellement »

$$\begin{array}{|c|c|c|c|c|} \hline 0 & 1 & 1 & 0 & 0 \\ \hline \end{array} = 12$$

Fitness Landscape (paysage évolutif) :
fonction performance sur l'espace de
recherche



Étapes de mise en œuvre d'un AG

- Choix « algorithmiques »
 - Représentation des individus (codage du génotype)
 - Génération de la population initiale
 - Modalités de reproduction (opérateurs d'évolution)
 - Choix du mode de sélection des individus
- Choix « paramétriques »
 - Taille de la population, taux de reproduction, de survie, de croisement, de mutation

Choix paramétriques ...

- Outre les choix « algorithmiques », la mise en œuvre d'un algorithme génétique demande de fixer un certain nombre de paramètres
- n : nombre d'individus dans la population
de l'ordre de la centaine (souvent fonction de la difficulté de calcul de la fonction de fitness)
- T_m : taux de mutation
- T_c : taux de cross-over

Initialisation de la population

- *Si possible*: tirage aléatoire uniforme dans l'espace de recherche
 - Codage binaire : 0 ou 1 avec une probabilité de 0.5
 - Codage à valeurs réelles : tirage uniforme dans un intervalle donné (applicable uniquement dans un ensemble borné)

Aperçu

- Introduction.
- **Représentation d'un problème.**
- Constituants de base des algorithmes génétiques : sélection, opérateurs d'évolution et remplacement.
- Quelques considérations.
- La programmation génétique.

Certains transparents présentés dans ce chapitre s'inspirent de ceux du Prof. J-C. Latombe (Université de Stanford) et ont été adaptés à notre contexte, ceci avec l'autorisation de l'auteur.

Représentation et initialisation

Exemple

- Un individu est codé par une chaîne de bits (génotype)
- Chaque chaîne code une solution candidate
- La population initiale est aléatoire

1	0	1	1	0
---	---	---	---	---

0	1	1	0	0
---	---	---	---	---

1	1	1	1	0
---	---	---	---	---

0	0	1	0	1
---	---	---	---	---

• • •

1	0	1	0	1
---	---	---	---	---

N individus générés aléatoirement
(population initiale)

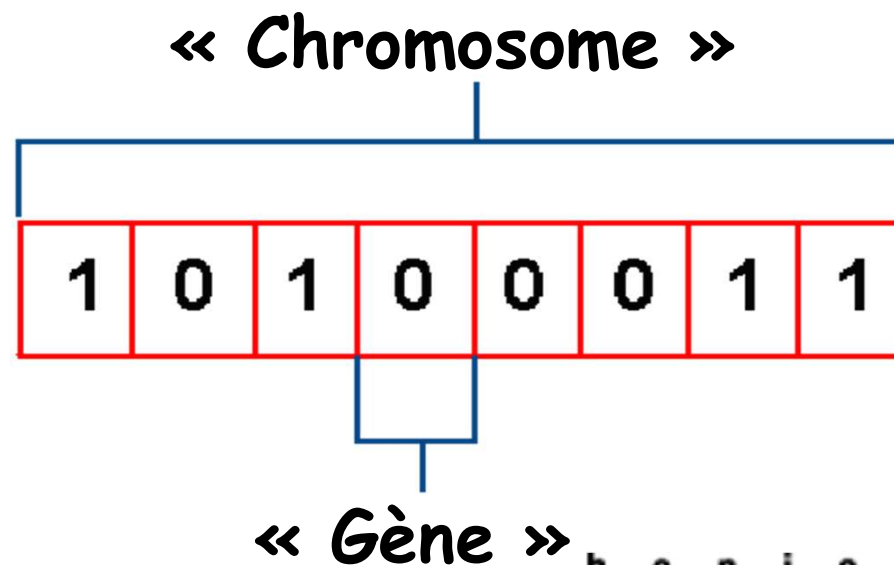
Choix de la représentation (génotype)

- Comment représenter un individu (phénotype ou solution candidate) par un génotype?
- Nombreuses possibilités, **mais**, le codage doit être adapté au problème posé.
- Choix de représentation dépend des autres étapes.
 - génotype sera converti en phénotype et évalué
 - génotype manipulé par des opérateurs génétiques (mutation, cross-over, ...)

✎ *Un codage inadapté peut compromettre l'évolution ...*

Représentations discrètes

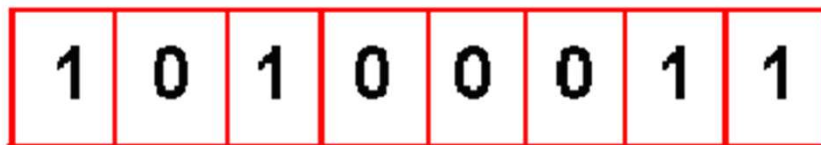
- Un individu peut être représenté par un ensemble de valeurs discrètes (codage binaire, entiers, ...).
- **Semble** très proche de la biologie (ACTG).
- Exemple : représentation binaire.



Représentations binaires

- Un génotype binaire peut conduire à une grande variété de phénotypes :

Génotype 8 bits



Phénotype

nombre entier

valeur « réelle »

ordonancement

...

Codage de « réels » à partir d'une représentation binaire

- Le codage de valeurs « réelles » est trivial si on connaît la dynamique des paramètres et la précision souhaitée.
 - Codage d'un « réel » entre -0.5 et 0.5 avec une précision de 0.001
 - Codage sur 10 bits ([0-1023]) et normalisation $((x/1023)-0.5)$
- Le codage binaire pose le problème des « falaises de hamming » (*hamming cliffs*).
 - Cette probabilité n'est pas liée à la proximité des phénotypes mais à la proximité des génotypes (distance de hamming)

⇒ *Un codage inadapté peut compromettre l'évolution ...*

Code de Gray (*Gray Coding*)

- Codage qui permet à 2 entiers adjacents d'être distants d'une seule mutation
 - Pas de falaises de Hamming
- Recette de traduction
entier binaire $\Rightarrow$ Gray
 - commencer à droite, copier le 1er bit
 - ensuite écrire 1 si le bit change et 0 sinon

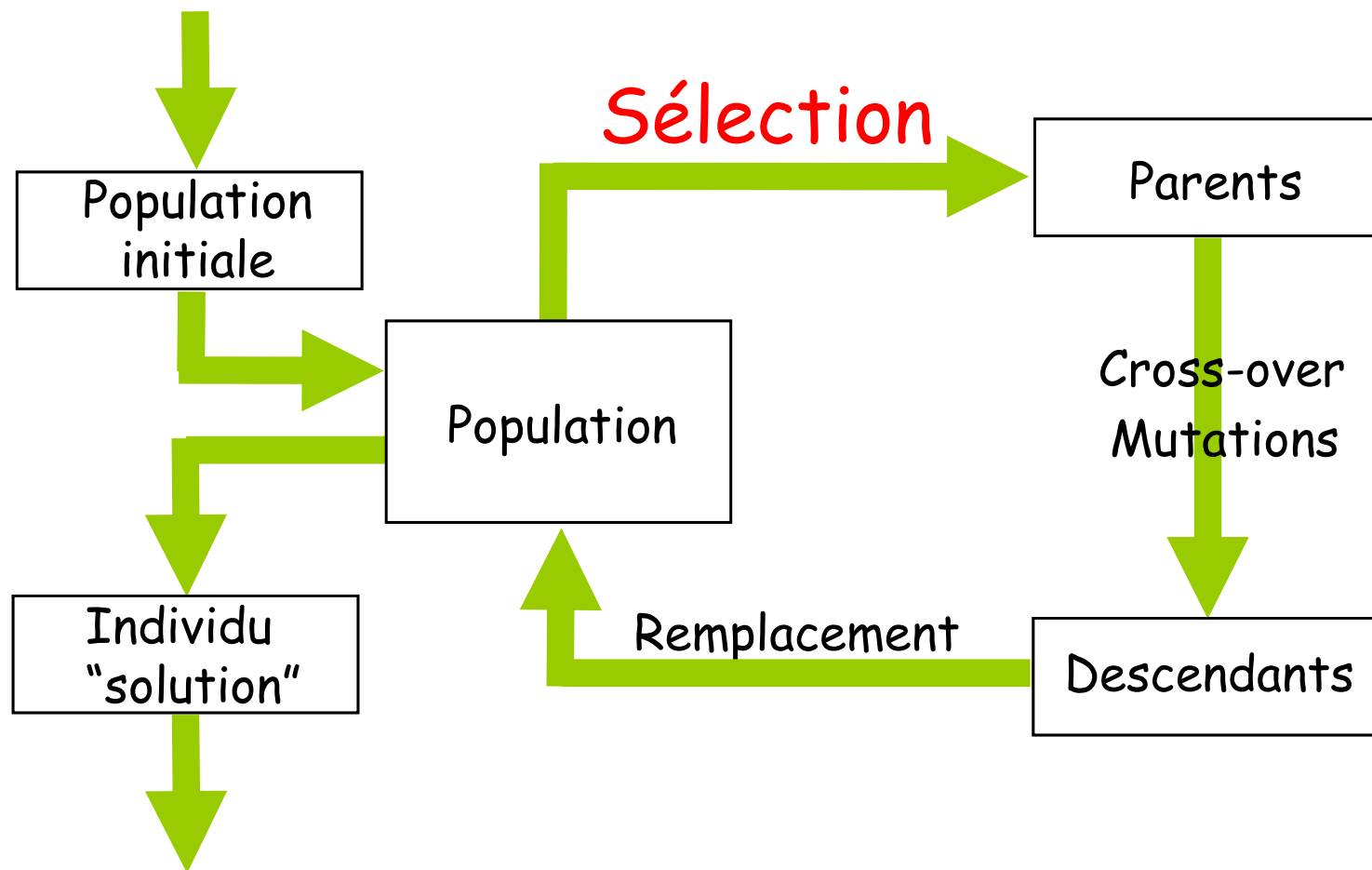
Exemple, nombre sur 3 bits :

Binaire	Valeur	Gray
000	0	000
001	1	001
010	2	011
011	3	010
100	4	110
101	5	111
110	6	101
111	7	100

Aperçu

- Introduction.
- Représentation d'un problème.
- **Constituants de base des algorithmes génétiques : sélection, opérateurs d'évolution et remplacement.**
- Quelques considérations.
- La programmation génétique.

Principe du cycle d'évolution

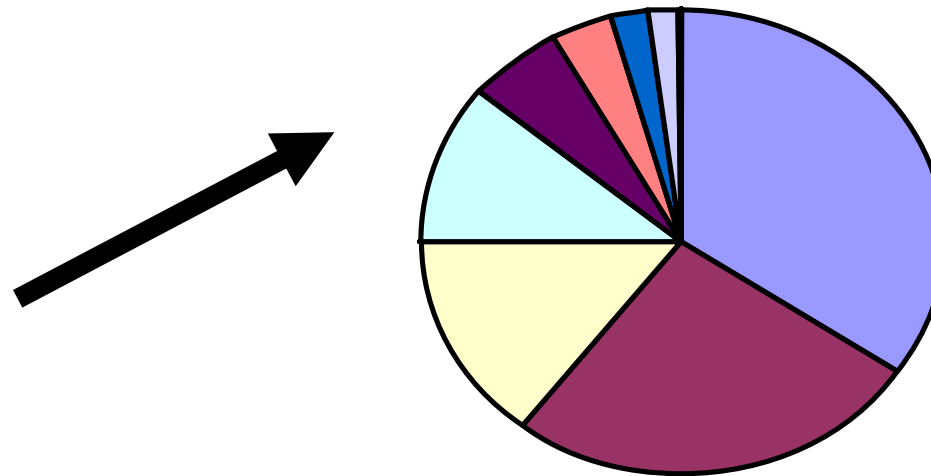
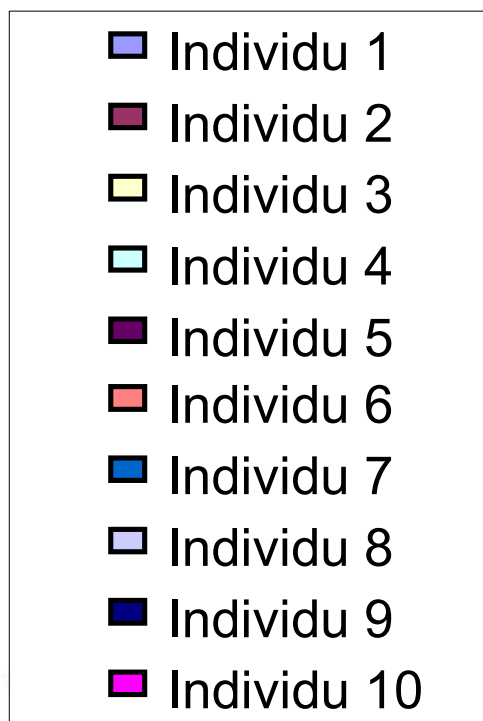


Stratégie de sélection « naturelle »

- But de la sélection naturelle :
 - Assurer que les meilleurs individus aient plus de chance d'avoir une descendance que les autres
- La « pression de sélection » doit guider l'exploration.
- Les moins bons individus doivent conserver une chance de se reproduire.
 - ils peuvent contenir des gènes intéressants.
 - => la pression de sélection doit préserver une part de bio-diversité

Sélection proportionnelle

- Un individu i de fitness f_i a une probabilité $f_i / \sum_j f_j$ d'être sélectionné comme parent
- Principe de la roulette biaisée
 - Chaque individu reçoit une portion de roulette proportionnelle à sa fitness



Sélection proportionnelle

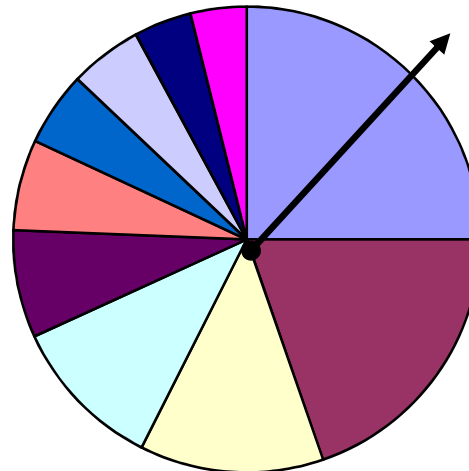
- Désavantages de la sélection proportionnelle :
 - Réduction rapide de la diversité
 - Risque de convergence prématurée (les individus les mieux adaptés vont rapidement phagocyter l'ensemble de la population)
 - La pression de sélection reste faible lorsque les fitness sont très similaires
 - Plusieurs solutions pour éviter la convergence prématurée (vers un optimum local)

⇒ *Normalisation de la fonction de fitness ...*

Sigma Scaling

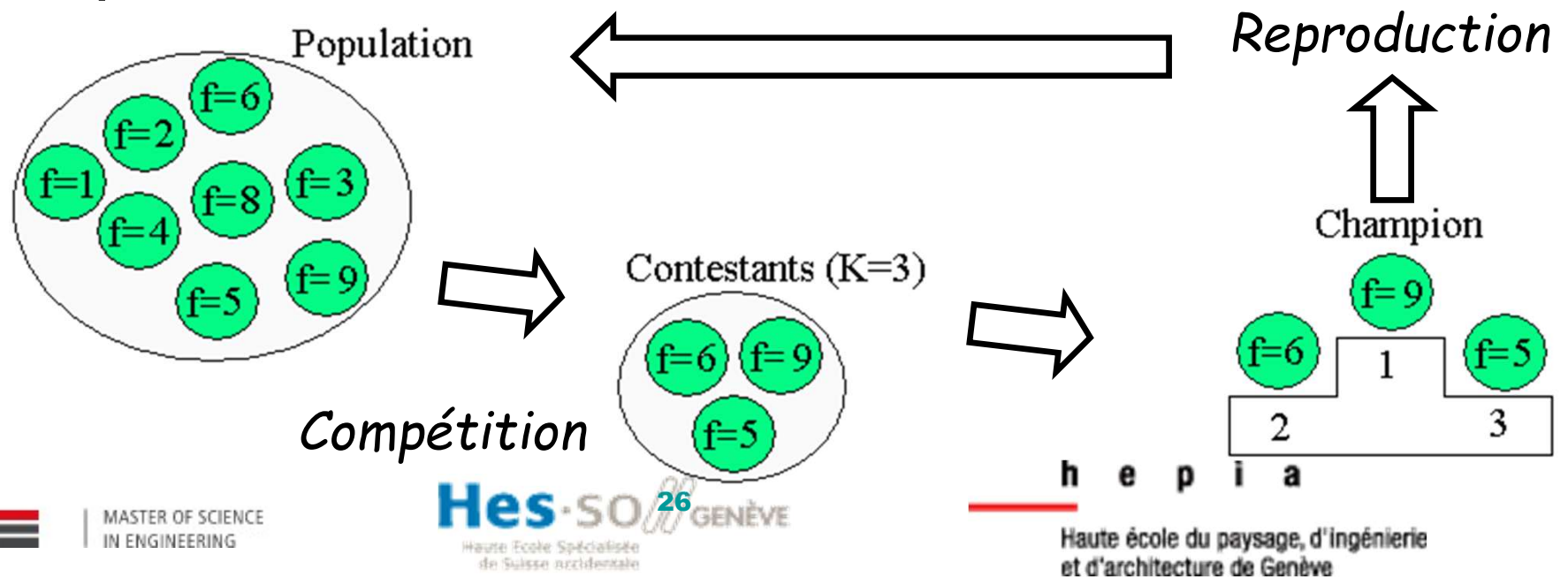
- But : éviter la perte de biodiversité si un individu est beaucoup mieux adapté que les autres.
 - La grande majorité des descendants sont issus d'un seul individu
 - Risque de convergence prématurée
- Normalisation basée sur la variance (sigma scaling).
 - Uniformisation de la pression de sélection

$$\begin{cases} \text{Si } \sigma \neq 0 & \text{Fit}(x) = 1 + ((f(x) - \text{Mean}(f)) / 2 * \sigma) \\ \text{Sinon} & \text{Fit}(x) = 1 \end{cases}$$

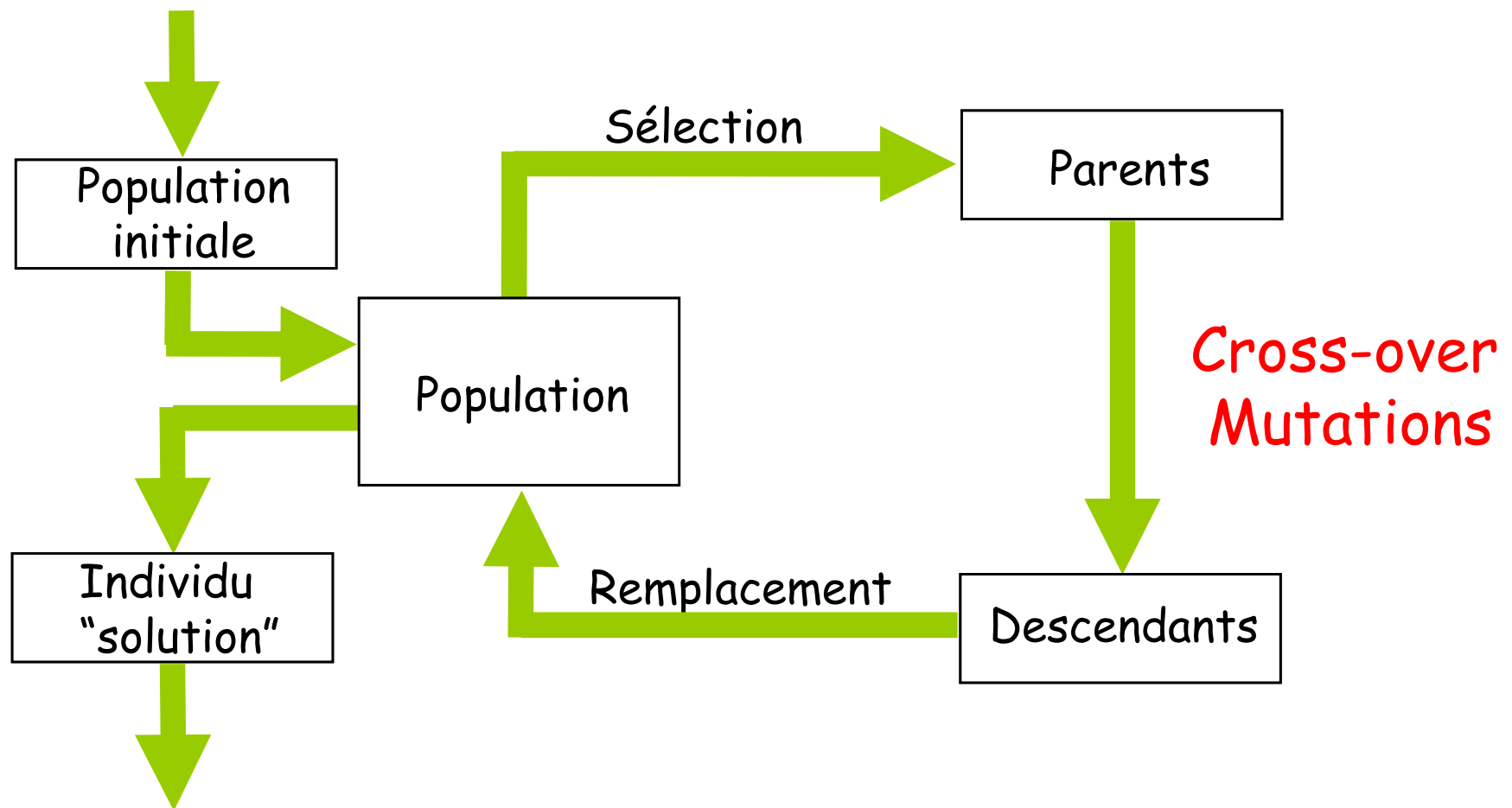


Sélection par compétition (*tournament*)

- Sélection basée sur un remplacement partiel de la population
 - Sélection aléatoire de k individus (sans remplacement),
 - Compétition entre les k individus,
 - Seul le meilleur se reproduit
- Exemple :

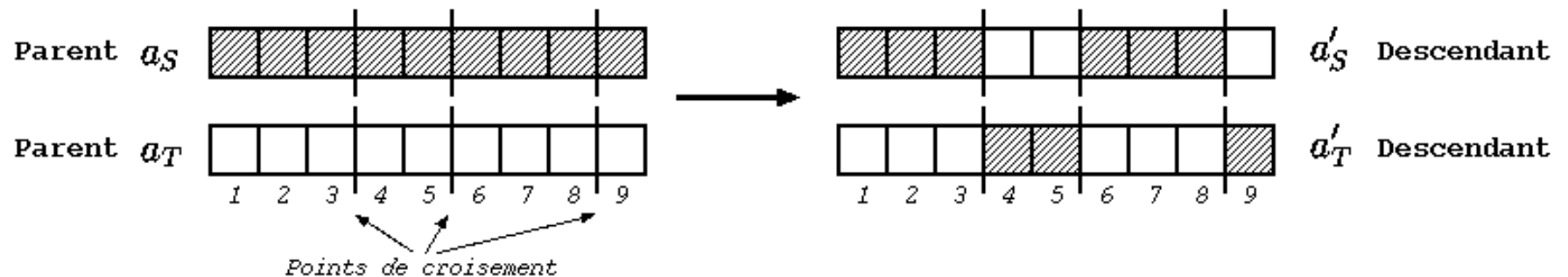


Principe du cycle d'évolution



AG : génération

- Sélection
- Croisement / recombinaison (*crossover*)
 - choix des individus formant la nouvelle population

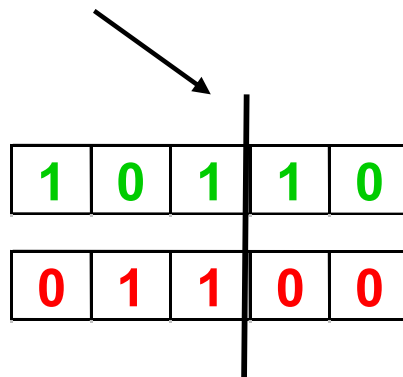


- Mutation
 - modification aléatoire d'un individu

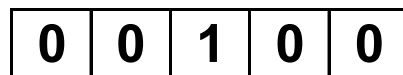
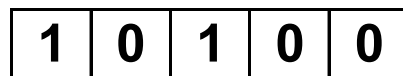
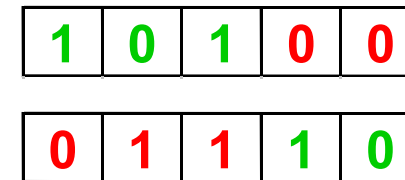
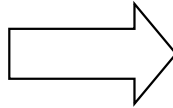


Croisement / mutation

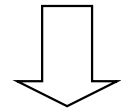
point de croisement



croisement



mutation



Intérêts relatifs de la mutation et du cross-over

- Cross-over
 - Permet de réexprimer des caractères déjà présents dans la population,
 - L'effet du cross-over diminue lorsqu'on se rapproche de la solution,
 - Opérateur d'**exploitation**
- Mutations
 - Indispensables pour éviter les maxima locaux,
 - Opérateur d'**exploration** ...

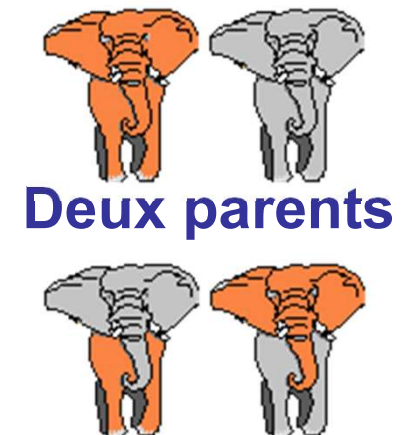
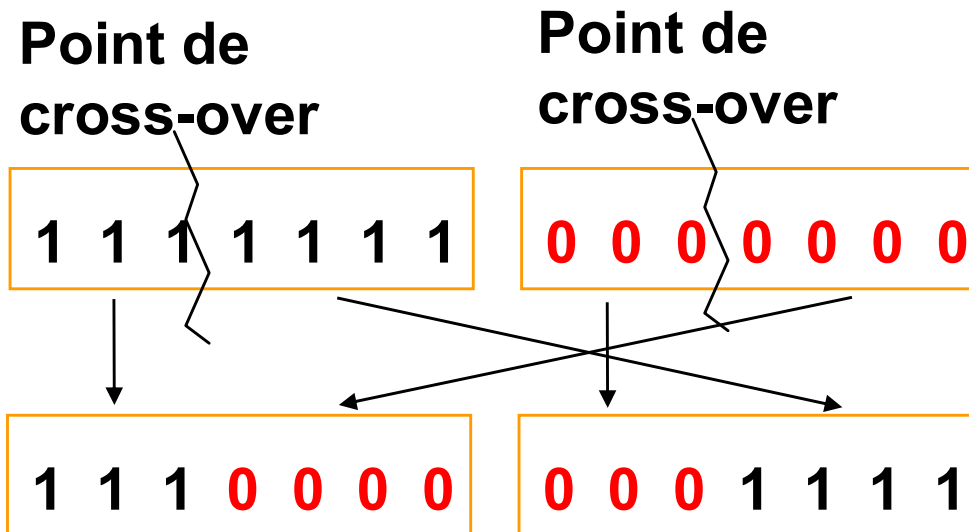
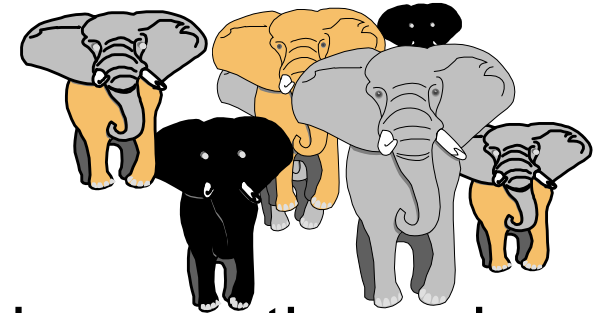
⇒ *La recombinaison est souvent plus difficile à mettre en œuvre ...*

Les opérateurs génétiques: le cross-over

- Il peut y avoir un ou plusieurs opérateurs de recombinaison (cross-over), mais :
 - les descendants doivent hériter de chacun des parents (sans quoi il s'agit d'une simple mutation)
 - les opérateurs de recombinaison doivent tenir compte de la représentation du génotype et de la fonction de transfert génotype/phénotype
 - Le cross-over est souvent appelé «Crossing-over» ou «recombinaison»
 - En particulier en biologie
- ✧ *Les recombinaisons doivent produire des chromosomes (i.e. des individus) valides (i.e. viables)*

Le cas des représentations binaires

- Deux parents donnent deux enfants
 - Parfois un seul
- Chaque chromosome est découpé en deux parties qui sont recombinaées pour former les descendants



Deux parents

Variantes pour le cross-over discret

- Cross-over multipoints
 - k points de cross-over tirés au hasard
- Cross-over uniforme
 - Les gènes sont échangés suivant une probabilité p_c

Représentation à valeurs réelles

- Si la solution recherchée est exprimée sous la forme d'une liste de valeurs réelles, le codage le plus naturel est d'exprimer le génome sous la forme d'une liste de réels
- Cas très courant couvrant un grand nombre d'applications (optimisation paramétrique)

⇒ *La manipulation individuelle, par des opérateurs génétiques, des bits d'un codage flottant n'a aucun sens ...*

Représentation à valeurs réelles

- Un individu X est un vecteur à valeurs réelles :

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, x_i \in R$$

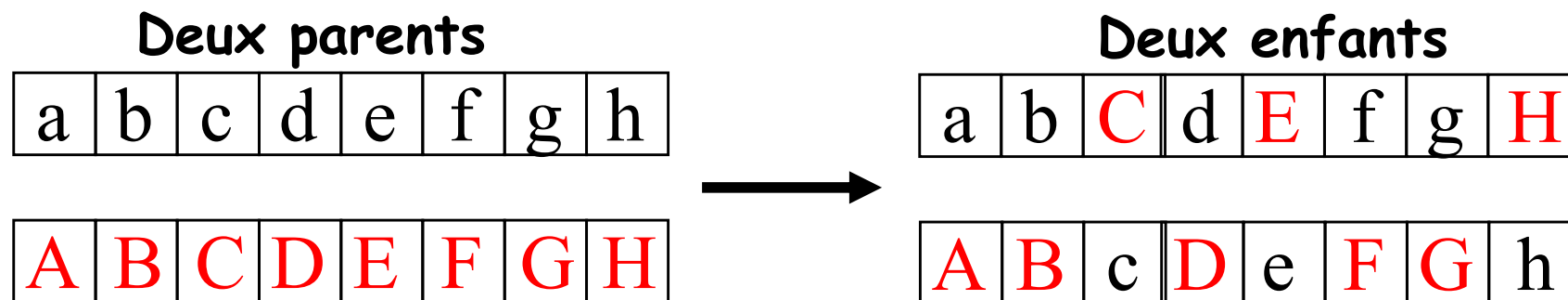
- La fonction de fitness f permet d'exprimer la qualité d'un individu (fonction de R^n dans R)

$$f : R^n \rightarrow R$$

Le cas des génomes à valeur « réelle »

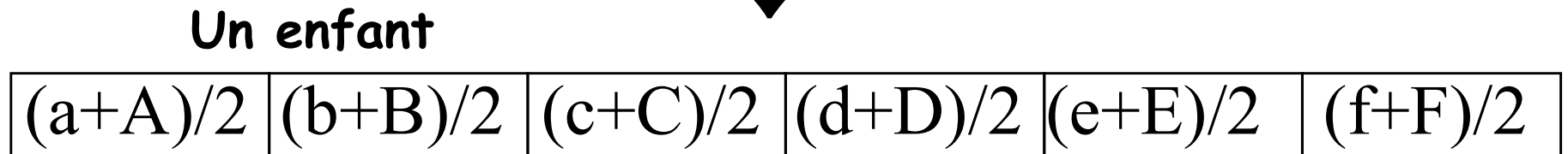
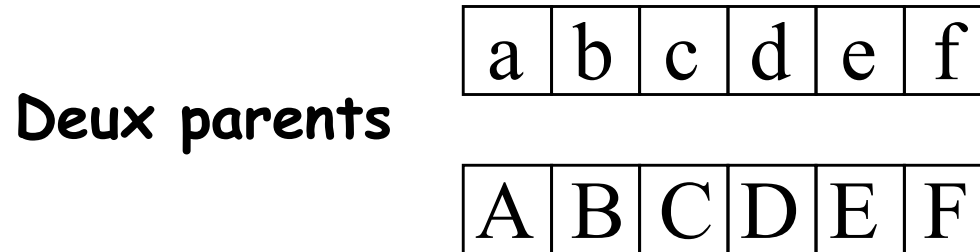
Recombinaison discrète :

- Recombinaison aléatoire des gènes à partir des deux parents
- Les lettres désignent des valeurs réelles



Le cas des génomes à valeur « réelle »

Recombinaison discrète :



Cas particulier : représentation ordonnée

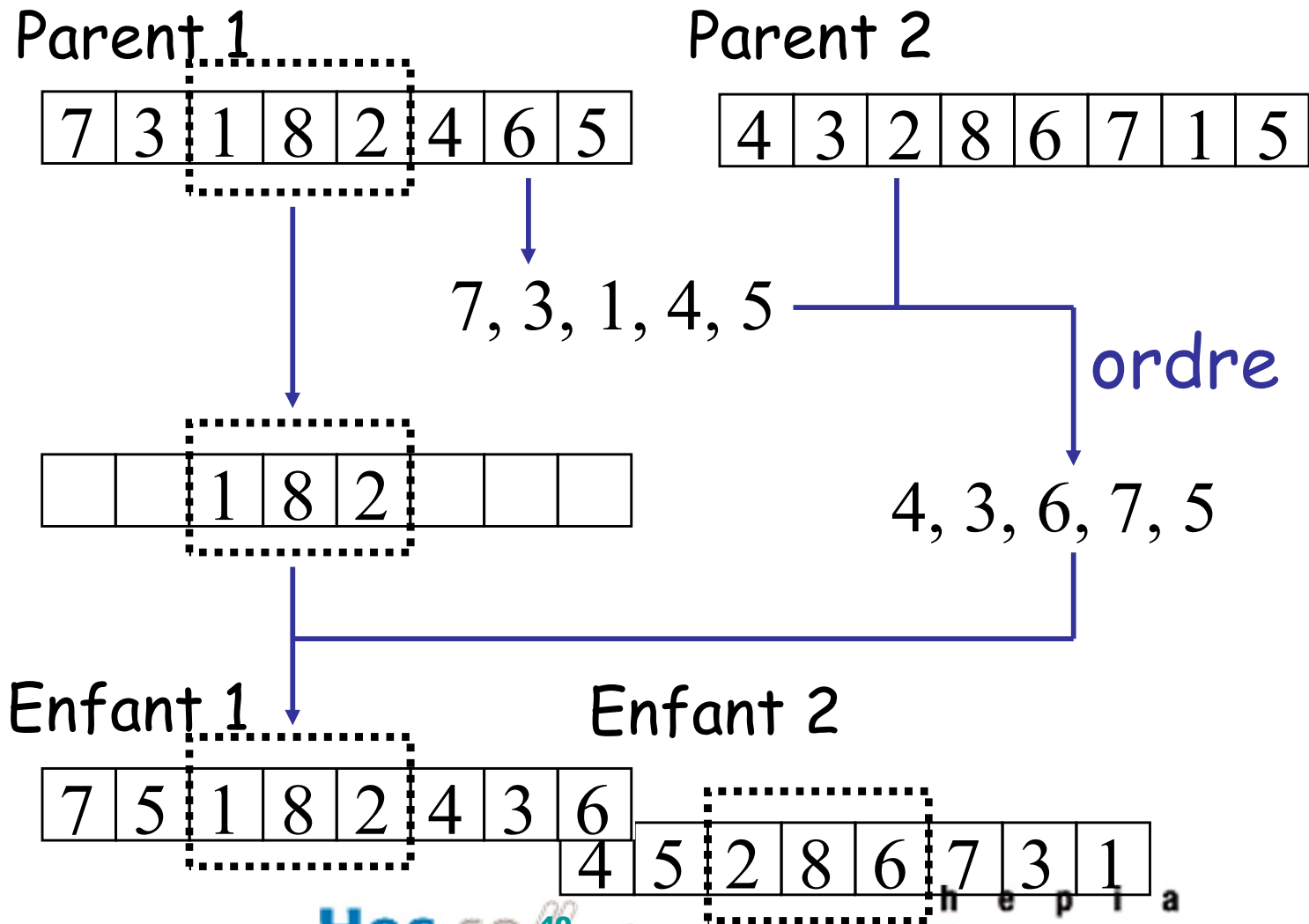
- Un individu est représenté par une liste de positions avec permutations
- Application : ordonnancement / séquençement
- Exemple : Le voyageur de commerce
 - Chaque ville reçoit un numéro d'ordre de 1 à n
 - Les solutions sont représentées par un ordre (ex : 5, 4, 2, 1, 3).

✚ *Les représentations ordonnées doivent être utilisées avec des opérateurs génétiques spécifiques assurant la validité des individus*

Le cas des représentations ordonnées (recombinaison)

- Choisir aléatoirement une partie du génome du premier parent et le recopier à l'identique
- Compléter le génome du descendant avec le deuxième parent
 - à partir du point suivant la partie recopiée
 - utiliser l'ordre des gènes du deuxième parent
 - balayage circulaire du chromosome
- Inverser le rôle des parents pour créer un deuxième descendant

Le cas des représentations ordonnées



Les opérateurs génétiques : la mutation

- Il peut y avoir un ou plusieurs opérateurs de mutation, mais :
 - un des opérateurs de mutation au moins doit permettre d'atteindre tous les points de l'espace de recherche
 - le taux de mutation est un facteur fondamental qui doit pouvoir être finement contrôlé

Mutation pour une représentation binaire

Génotype

Phénotype

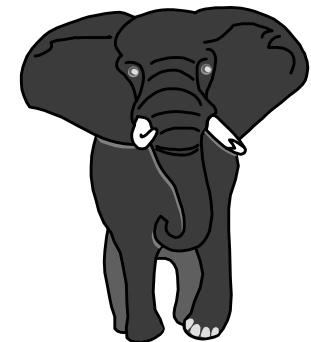
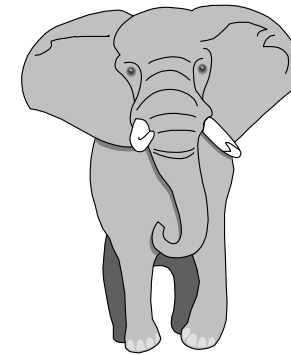
Avant

1 0 1 1 1 1 1

Après

1 0 1 0 1 1 1

↑
Gène mutant



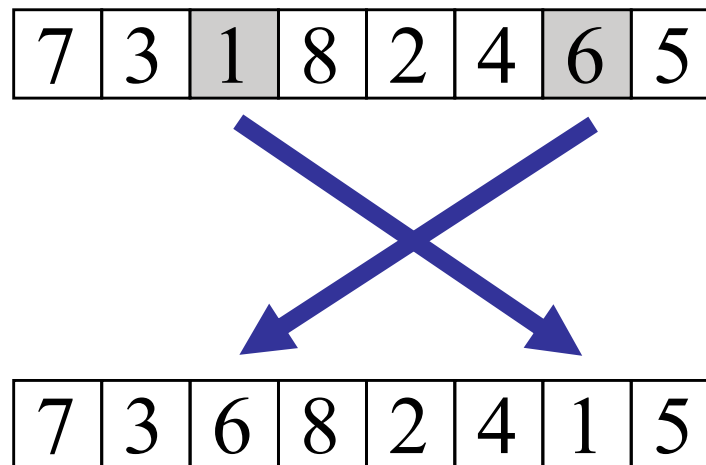
⇒ Les mutations apparaissent avec une probabilité p_m pour chaque gène (taux de mutation).

Le cas des génomes à valeurs réelles

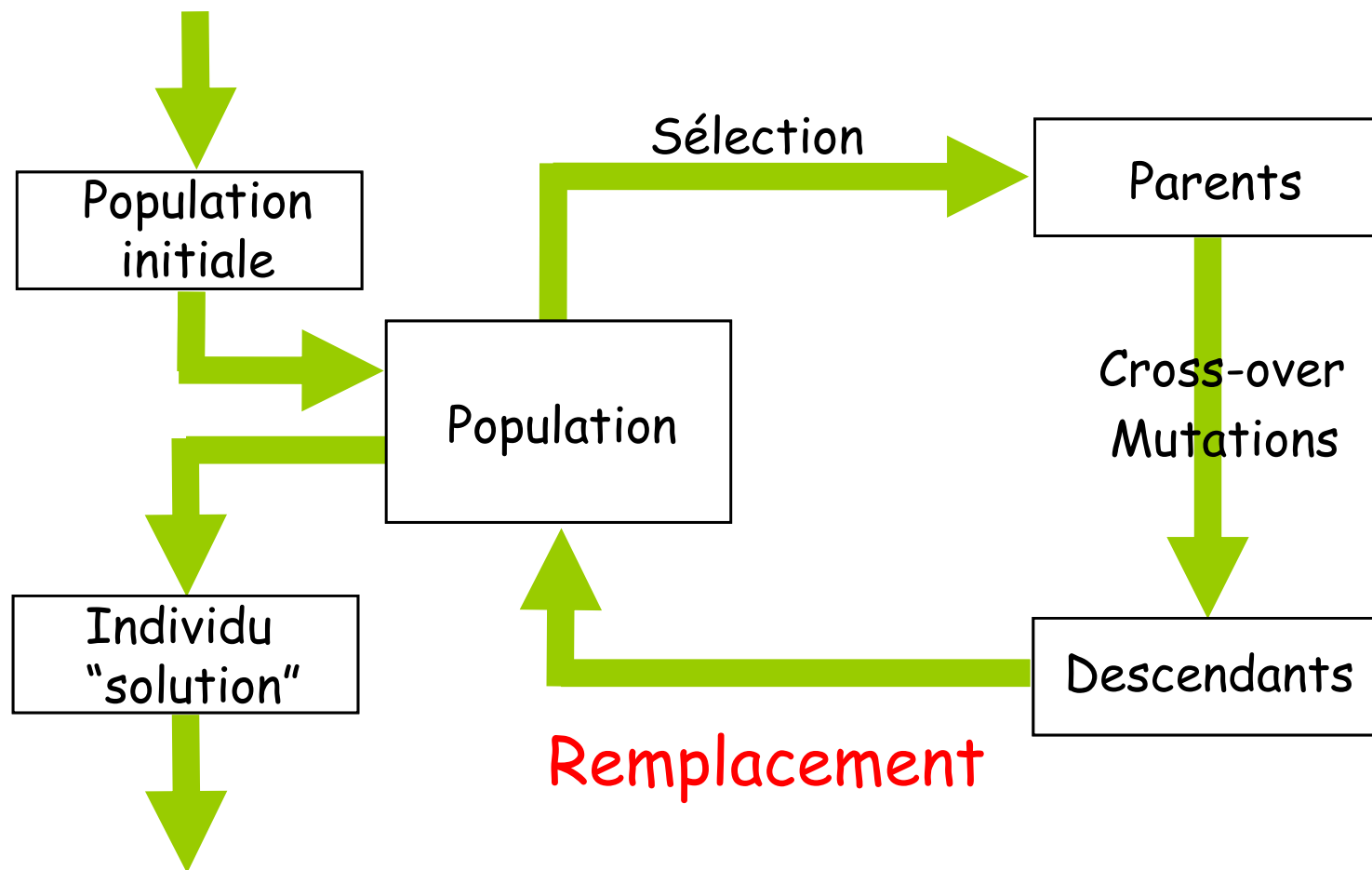
- Perturbation des gènes par addition d'un bruit aléatoire
- En général on utilise un bruit Gaussien obtenu par une loi normale $N(0, \sigma)$
 - 0 est la valeur moyenne du bruit
 - σ est l'écart type
- Soit, pour chaque paramètre : $x'_i = x_i + N(0, \sigma_i)$
- L'utilisation d'un bruit à distribution uniforme est possible

Le cas des représentations ordonnées (mutation)

- Échanger deux gènes choisis aléatoirement
(*gene swap*)



Principe du cycle d'évolution



Stratégie de « remplacement » (mortalité)

- Outre le critère de sélection, la pression de sélection est fonction de la « mortalité » des individus.
- En AG, la mortalité est généralement liée au « remplacement » (population constante).
- Deux approches :
 - Stratégies stochastiques
 - Remplacement déterministe

⇒ *Élitisme : conservation systématique du (des) meilleur(s) individu(s) ...*

Autres stratégies de remplacement

- Si N parents génèrent N enfants et sont tous remplacés, la sélection se fait uniquement sur le choix des parents.
- Mélanger parents et enfants puis sélectionner pour le remplacement.
- Générer plus d'enfants puis les sélectionner.

Intérêt de l'élitisme

- Comment assurer une croissance permanente du fitness ?
 - Élitisme : ré-introduire le ou les meilleur(s)
 - Préservation : « niche » pour le meilleur individu
- L'élitisme présente aussi un risque
 - Convergence prématurée due à la conservation d'un individu qui « pollue » en permanence la population

⇒ *Intérêt principal de l'élitisme : éviter la frustration de l'utilisateur ...*

Evaluation des individus

- Pour les applications réelles, cette étape est la plus coûteuse.
 - Difficulté de conception, temps de calcul, prix ...
 - Ne pas évaluer de nouveau les individus identiques ou non modifiés lors de leur reproduction
 - Plusieurs méthodes en fonction du problème :
 - Évaluation directe (fonction de fitness),
 - Simulation (boite noire),
 - Processus externe (robot, tests en situation, ...)
- ⇒ *Pour accélérer le processus il est possible d'utiliser une évaluation approchée ... au début du processus d'évolution ...*

Critère d'arrêt

- Difficile à déterminer puisque la qualité de la solution s'améliore de façon quasi-continue.
 - Plusieurs « solutions »
- Le maximum est atteint.
 - Suppose qu'il soit connu !
 - Il s'agit souvent d'un maximum *utile* ...
- Limite de temps de calcul.
 - On ne peut pas toujours évaluer autant d'individus qu'on le souhaite
- Limite de patience de l'utilisateur.
 - Lorsque rien n'évolue pendant plusieurs générations

Aperçu

- Introduction.
- Représentation d'un problème.
- Constituants de base des algorithmes génétiques : sélection, opérateurs d'évolution et remplacement.
- **Quelques considérations.**
- La programmation génétique.

Performance des algorithmes génétiques

- Du point de vue théorique :
 - Ne jamais tirer de conclusion à partir d'un test unique !
 - Les performances doivent être estimées statistiquement
 - Donc à partir d'un certain nombre d'essais

Point clef de l'algorithmique génétique

- Maintien de la diversité génétique au cours de l'évolution.
 - Il faut maintenir des caractéristiques génétiques différentes dans la population (bio-diversité)
 - Lorsqu'on perd la diversité génétique, tous les individus deviennent semblables
 - Effet boule de neige
 - Convergence vers l'optimum local le plus proche
- ✧ *En théorie, les mutations permettent de continuer à explorer l'espace ... en pratique, la perte de diversité génétique est irréversible ...*

Point clef de l'algorithme génétique

- Dilemme Exploration / Exploitation

- **Exploration** = tester les zones inconnues

- Trop d'exploration revient à une marche aléatoire et compromet la convergence

- **Exploitation** = essayer d'améliorer le meilleur individu trouvé jusqu'ici

- Trop d'exploitation revient à une recherche locale et conduit à une convergence vers un optimum local

⇒ *Dilemme exploration / exploitation*

Avantages des AGs

- Bon rapport coût / résultat sur une grande classe de problèmes.
- Parallélisme intrinsèque.
- Robuste, tolérant aux fautes.
- Applicable sans connaissance préalable du domaine d'application.
- Simple à programmer.
- Attention, « everything is problem dependent »
 - Les algorithmes génétiques ne sont pas la panacée universelle
- les AGs sont particulièrement adaptés lorsque le problème :
 - contient beaucoup de données / de paramètres
 - contient des paramètres interdépendants (problème complexe)
 - comporte des optimums locaux

Inconvénients des AGs

- Pas de garantie de convergence en un temps fini.
- Faiblesse des fondements théoriques et mathématiques.
- Souvent gourmands en calcul donc lents (mais aisément parallélisables).
- Chaque individu doit être évalué, même les individus inadaptés.

✧ *Le comportement de l'algorithme dépend d'un grand nombre de paramètres qui peuvent être difficiles à fixer*

Aperçu

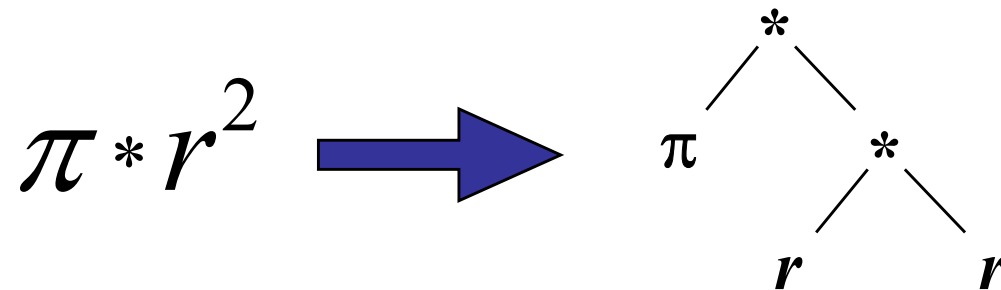
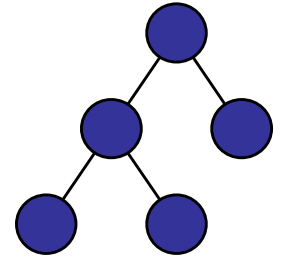
- Introduction.
- Représentation d'un problème.
- Constituants de base des algorithmes génétiques : sélection, opérateurs d'évolution et remplacement.
- Quelques considérations.
- **La programmation génétique.**

La programmation génétique

- Proposée par John Koza en 1989
 - A rapidement conquis le milieu des algorithmes évolutionnistes
- Permet d'utiliser l'évolution pour construire un programme
 - Aspect quasi-universel
- Basée sur un génome arborescent
 - La taille du génome est variable
 - Moins figé que les algorithmes génétiques classiques

Représentations arborescentes

- Les individus sont des arbres (généralement binaires).
- Chaque arbre est constitué de fonctions et de valeurs terminales codant une expression.
- Les fonctions et les valeurs terminales sont choisies parmi deux ensembles prédéfinis :
 - fonctions : sinus, cosinus, addition, soustraction, et, ou, if-then-else, while, avancer, tourner
 - valeurs terminales : X, Y, 0.456, vrai, faux, r



Représentations arborescentes

- La représentation arborescente est dépendante du choix des ensembles de fonctions et de valeurs terminales.
- Il est souhaitable que ces ensembles soient **clos** et **suffisants** :
 - **clos** : toutes les fonctions doivent pouvoir accepter comme arguments tous les éléments de l'ensemble des valeurs terminales ou tous les types retournés par les fonctions.
 - **suffisants** : il doit exister au moins une solution dans l'espace des programmes constructibles à partir de l'ensemble des fonctions et des valeurs terminales.

Construction de la population initiale

- Choisir aléatoirement une fonction f dans l'ensemble des fonctions F . Cette fonction est la racine de l'arbre/individu.
- Pour toute fonction ayant un nombre fixe d'arguments (fonctions unaires, binaires, ternaires, ..., n -aires), choisir pour chacun de ses arguments une fonction ou une valeur terminale
 - si le choix porte sur une valeur terminale, elle devient une feuille de l'arbre
 - si le choix porte sur une fonction, reprendre le processus récursivement (racine du sous-arbre)

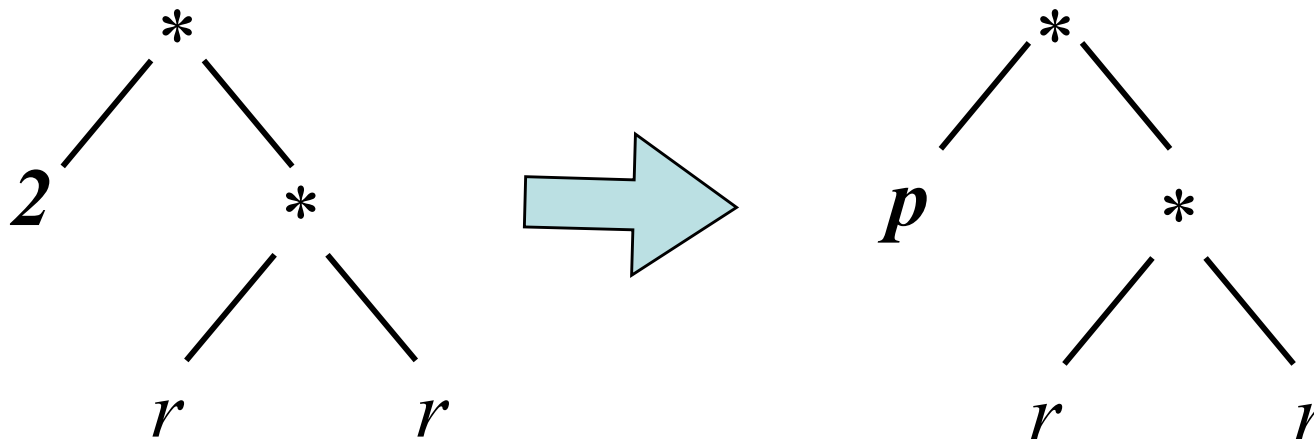
⇒ *Une profondeur maximale permet d'assurer la convergence du processus*

Construction de la population initiale

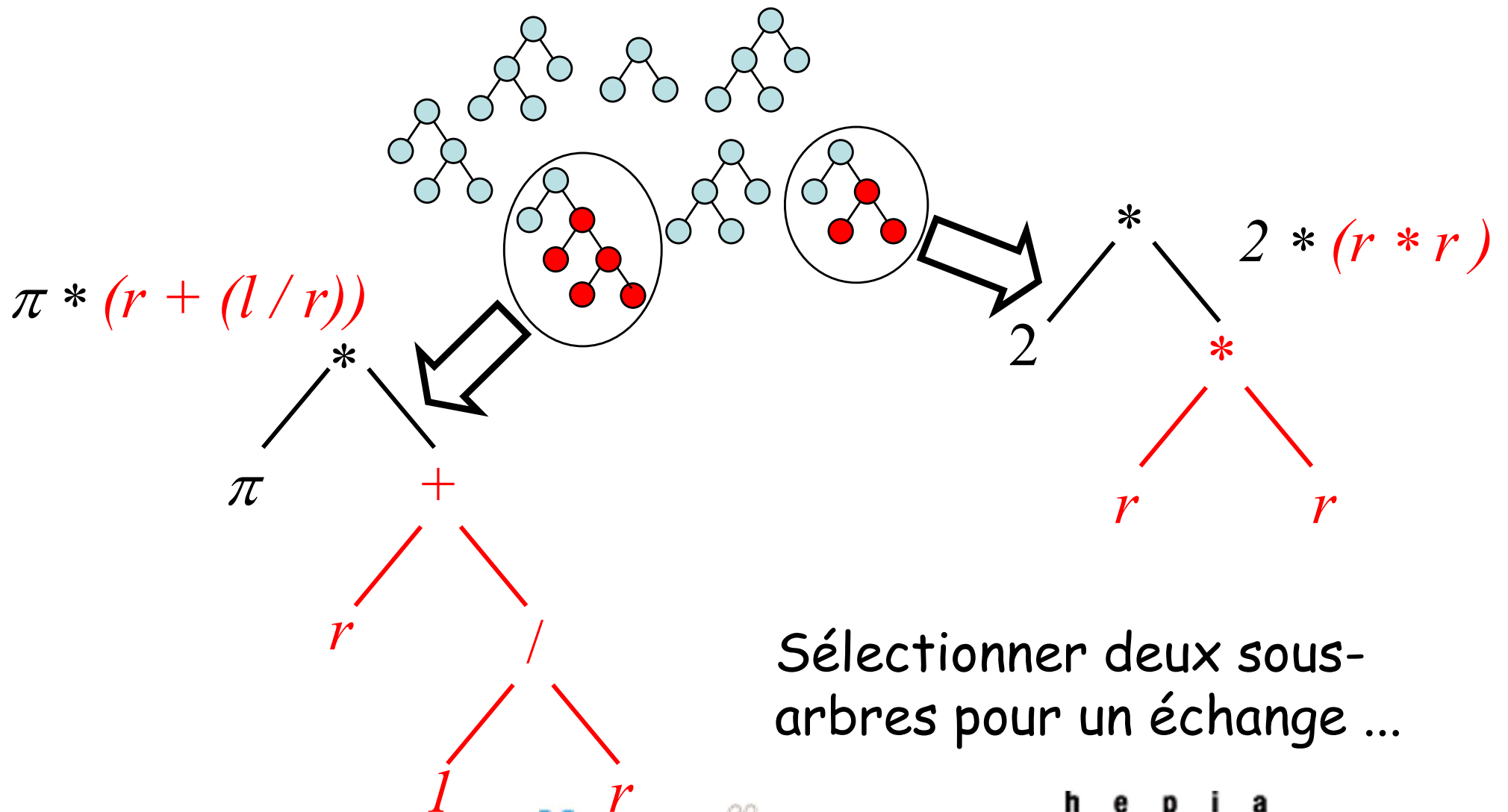
- Construction complète :
 - tous les chemins sont de longueur égale
 - les valeurs terminales ne sont sélectionnées qu'à la profondeur maximale. Pour tous les nœuds intermédiaires, seules des fonctions sont choisies
- Construction croissante :
 - choix aléatoire à tous les niveaux de l'arbre (sauf à la profondeur maximale où seules des valeurs terminales peuvent être placées)
- Construction « Ramp-half-and-half » :
 - la profondeur maximale est variable entre les individus. Pour chaque profondeur, deux individus sont créés (un par la méthode complète et un par la méthode croissante)

Mutation

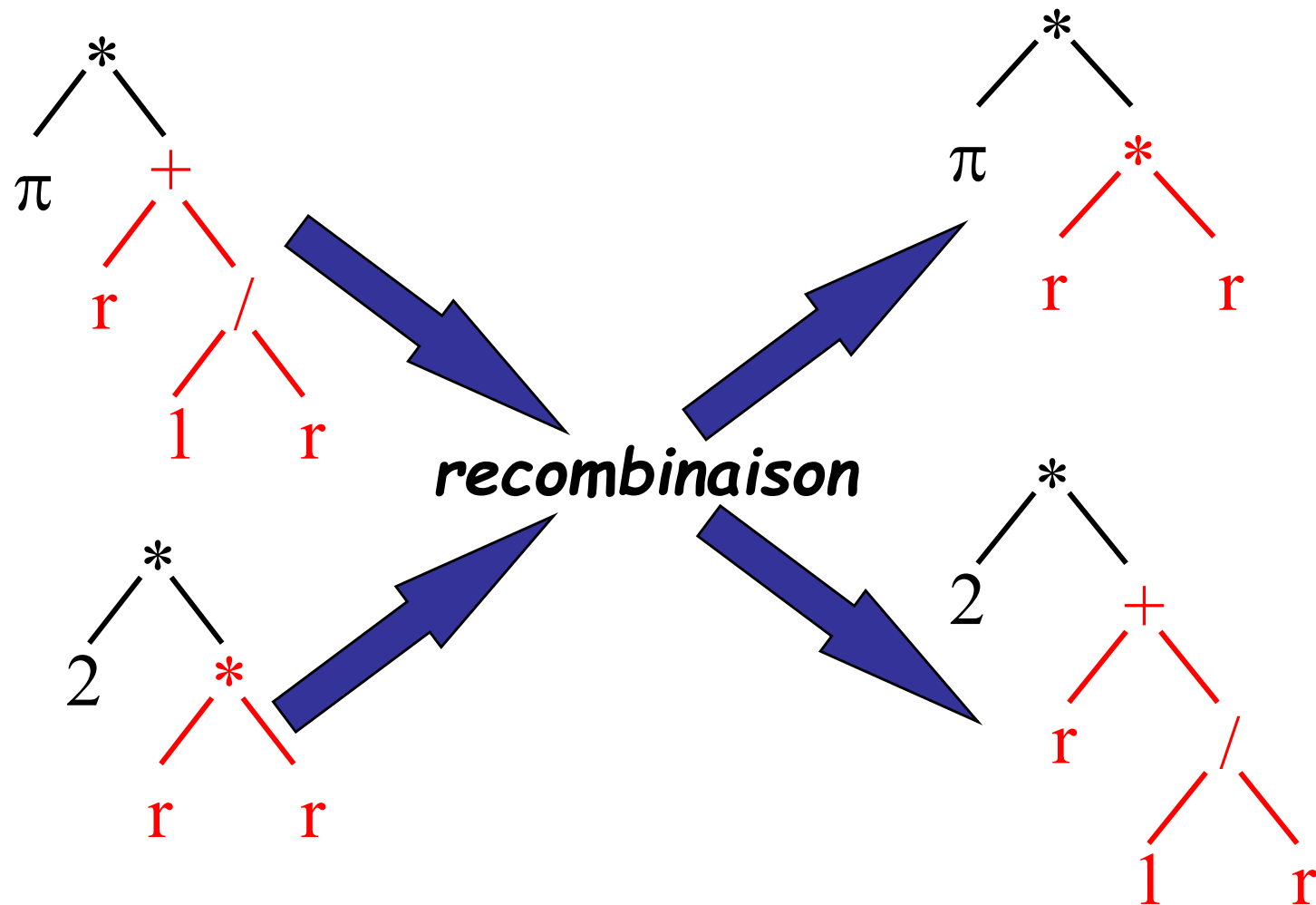
- Mutation « mono-point » : sélectionner un nœud au hasard et le remplacer par un nœud similaire



Recombinaison



Recombinaison

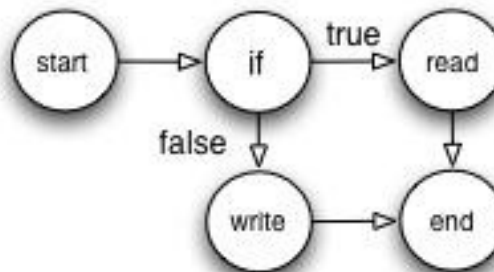
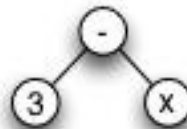


Autres représentations

Structure des individus

- linéaire
- arbre
- graphe

ld	r1,\$300000
mov	r1,r2
sub	r2,\$300000
sto	r2,r1



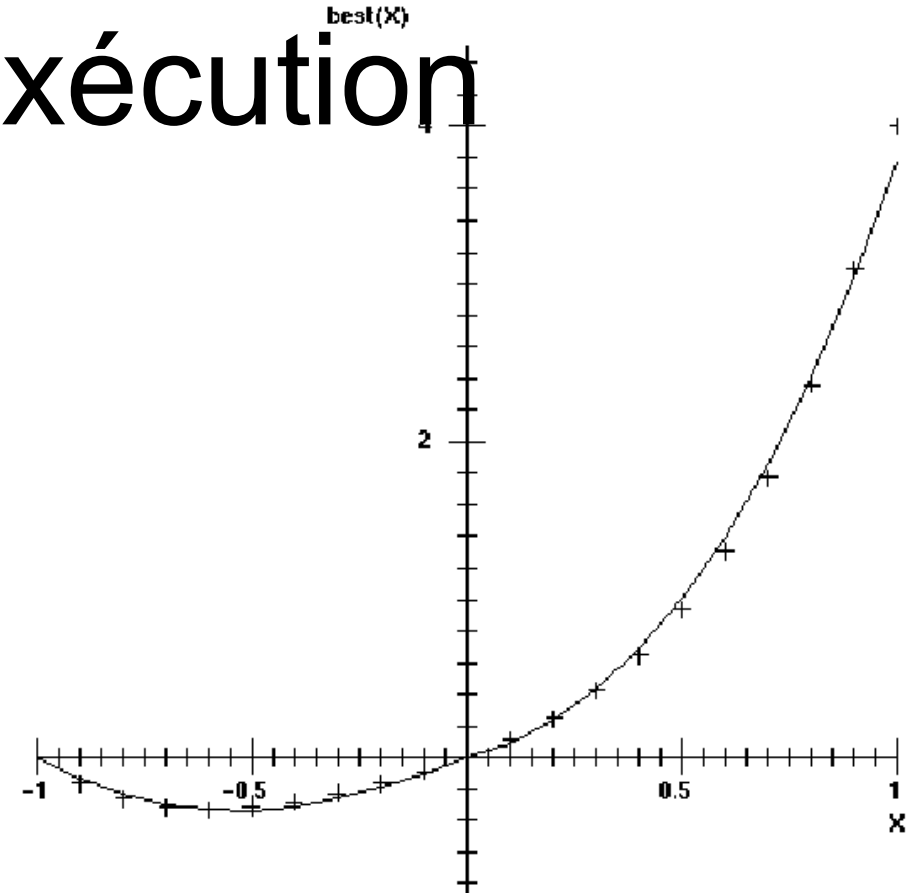
Quelques exemples

- Problème du sac-à-dos (*KnapSack*)
 - individu de taille variable
- Régression symbolique
- Vie artificielle
 - **creatures**
 - projet **golem**

Régression symbolique

exemple d'exécution

Meilleur programme génération 26

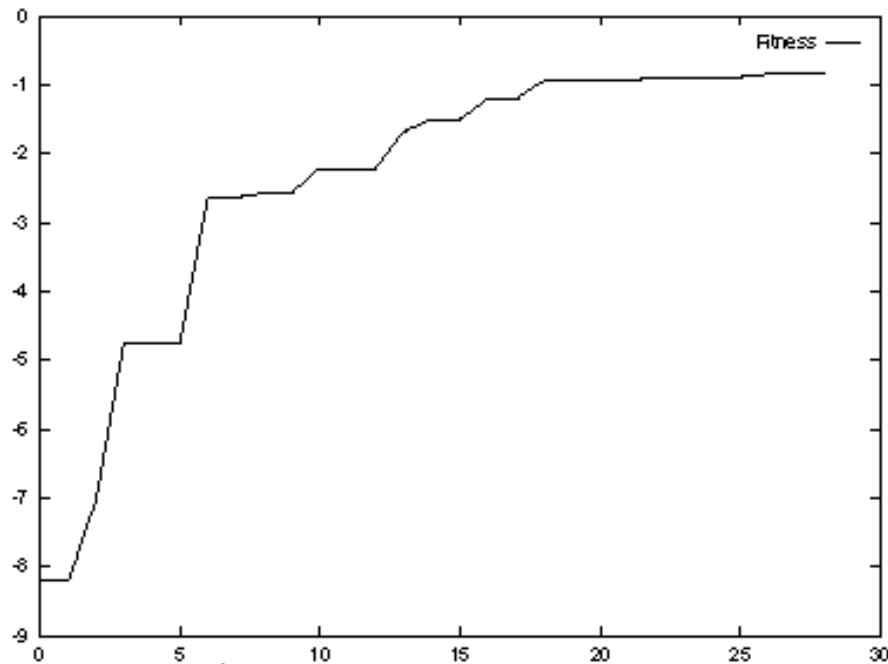


```
(* (+ (+ (+ (* (+ (pexp (plog x)) (sin (plog (+ x x)))) x)
(pexp (plog x)) (pdiv x x)) x) (plog (pexp (+ (plog (- (sin
(+ (plog (pexp (plog (- (sin (+ (+ (+ x x) (pdiv (* (+ (+ (+ x
(pexp (plog x)) (pdiv x x)) x) x) x)) x)) (pexp x)))))) x))
(pexp (plog (- (sin (+ (+ (+ x x) (pdiv (* (+ (+ (+ x (pexp (plog x))
(pdiv x x)) x) x) x)) x)) (pexp x)))))) (plog (- x x))))))
```

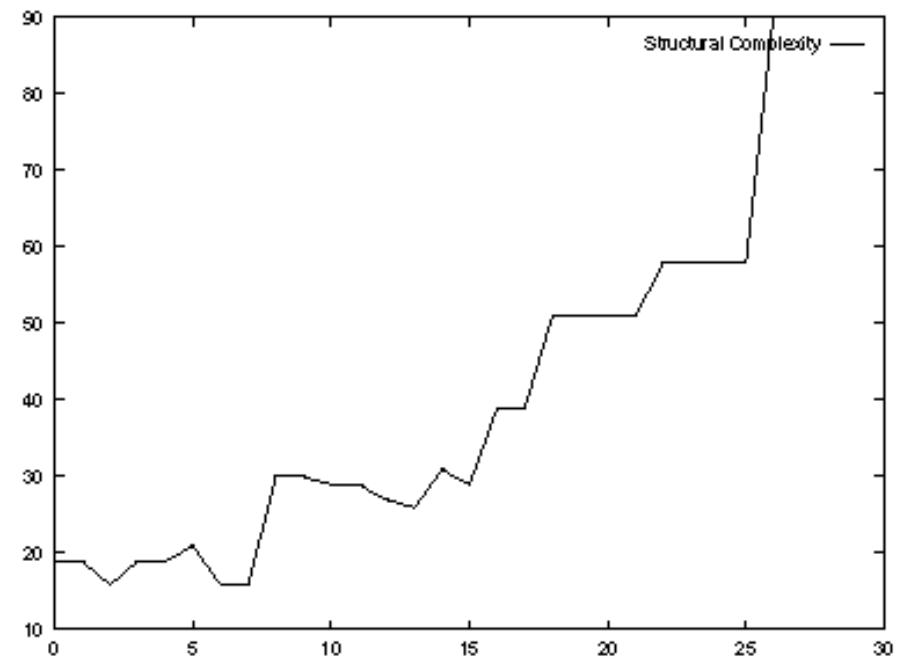
Régression symbolique

Évolution fitness et taille

Meilleur de la génération
performance(génération)

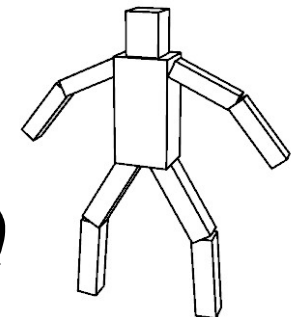
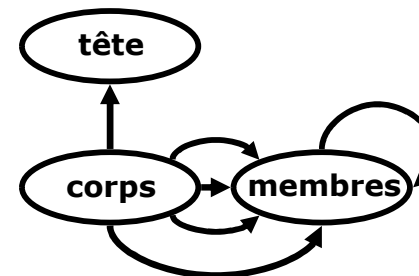
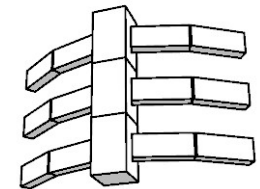
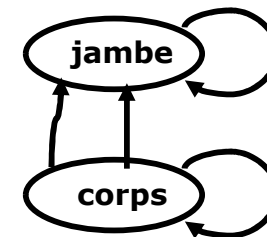
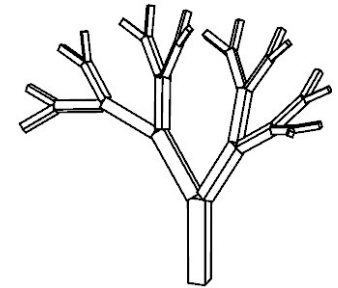
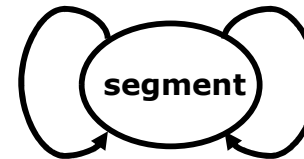


Meilleur de la génération
taille(génération)



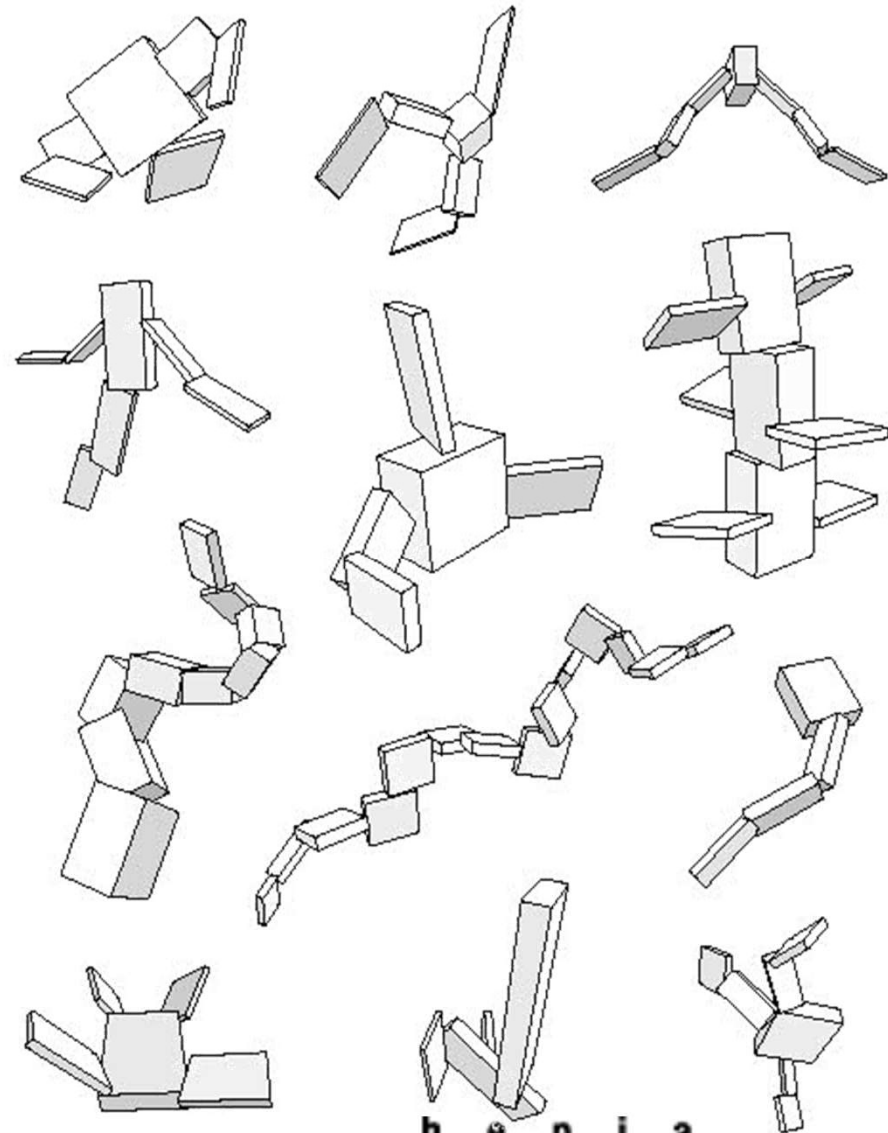
Evolved Virtual Creatures

- Chaque « créature » est définie par un graphe
 - Un nœud = un « bloc corporel »
 - Un lien = une articulation
 - Liens récurrents = structures répétitives
- Les nœuds et les liens sont valués
 - Dimensions
 - Limites articulaires
 - position/orientation
 - limites de récursivité
 - contrôle de l'articulation

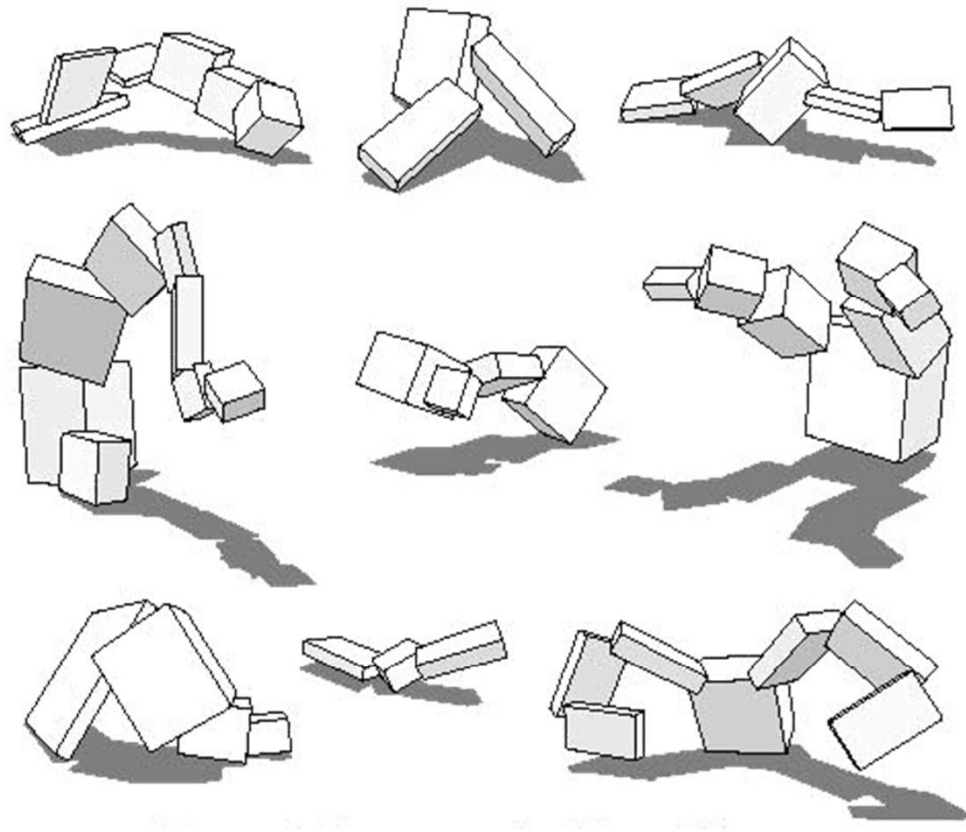


Evolving Virtual Swimmers (Karl Sims)

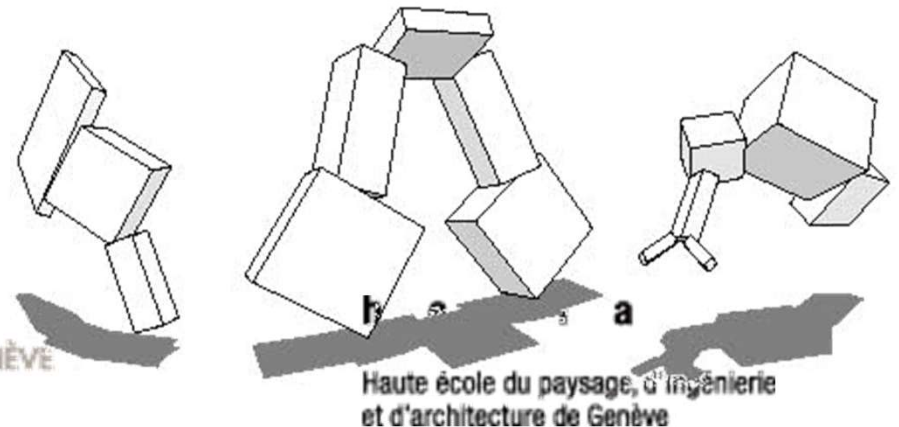
- Environnement :
 - Visqueux sans gravité
- Fitness :
 - Distance parcourue
- Trois types de formes :
 - Rames + gouvernail
 - Nageoires symétriques
 - « serpents marins »



Evolving Virtual Walkers/Jumpers (Karl Sims)



- Environnement :
 - « Grave » sans viscosité
- Fitness :
 - distance parcourue
 - hauteur atteinte
- Formes très variées :
 - déplacement basé sur le saut
 - reptation
 - « début de bipédie »



Evolved Virtual Creatures : Architecture

- Chaque créature « existe » et « vit » dans un environnement simulé
 - Le plus complexe reste la simulation du monde physique
 - Chocs
 - Frottements
 - Gravité
 - Viscosité
 - Capteurs
 - D'après Sims, toute erreur de simulation est rapidement exploitée par les créatures
- ⇒ La morphologie et le comportement des créatures sont fortement déterminés par les conditions environnementales

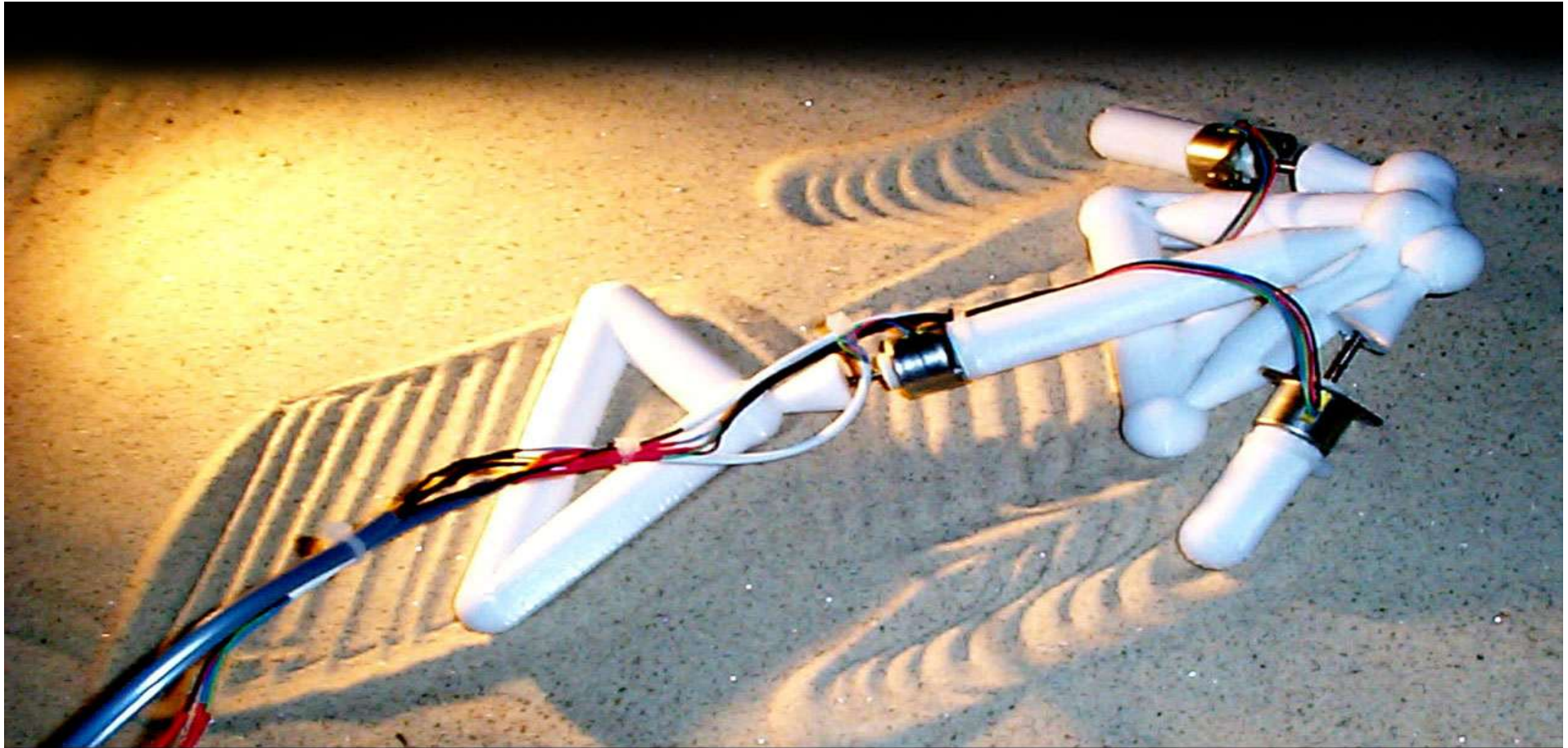
Evolved Virtual Creatures : Architecture

- La « population » est initialisée avec des graphes « aléatoires »
- Chaque graphe est « traduit » en créature
- Chaque créature est « évaluée » dans un environnement pré-déterminé (la physique de l'environnement est stable!) en fonction d'un critère de « fitness »
- Les individus les plus adaptés sont reproduits, moyennant quelques erreurs/imprécisions dans le processus de recopie
- Évolution sur plusieurs (dizaines de) générations

➤ Seules les « meilleures » créatures
sont visualisées

The Golem Project

Automatic Design and Manufacture of Robotic Lifeforms
(Hod Lipson et Jordan Pollack)



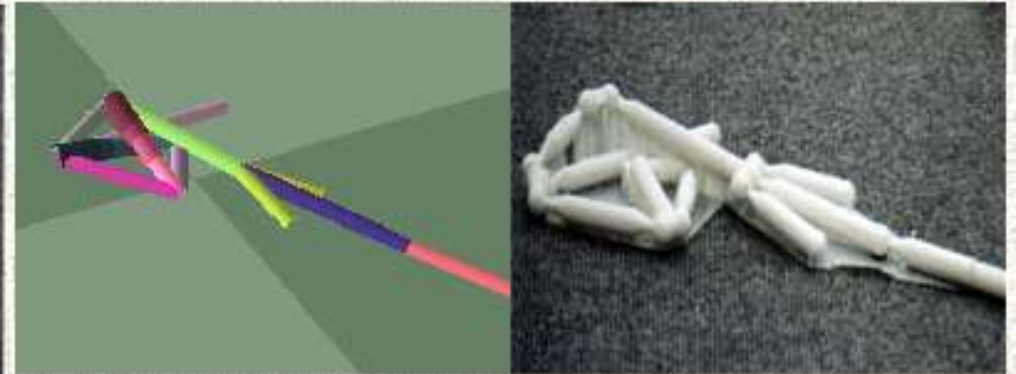
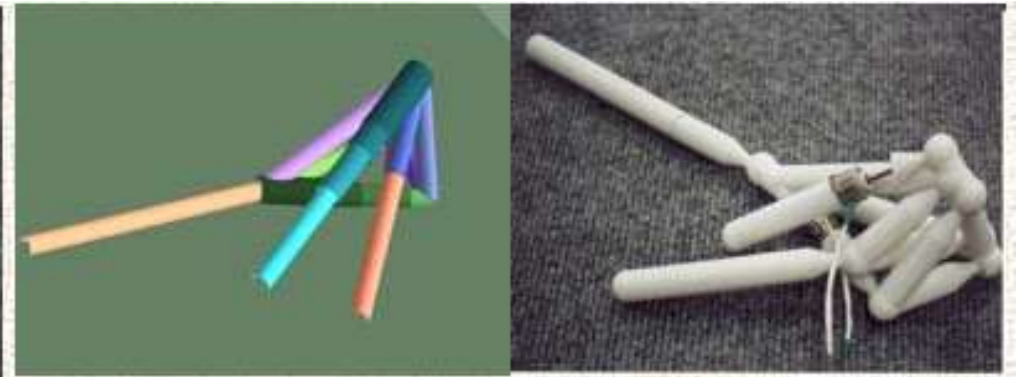
Golem Project Evolved Creatures

(Hod Lipson et Jordan Pollack)

Arrow



Ratchet



Tetra

Snake